

第 9 章：从 TRPO (Trust Region Policy Optimization, 信赖域策略优化) 到 PPO (Proximal Policy Optimization, 近端策略优化)

9.1 本章符号说明

符号/缩写	English	中文	具体含义
TRPO	Trust Region Policy Optimization	信赖域策略优化	在限制新旧策略 KL 距离的条件下优化代理目标。
PPO	Proximal Policy Optimization	近端策略优化	用 clipping 或 KL penalty 近似限制策略更新。
KL	Kullback-Leibler Divergence	KL 散度	衡量两个概率分布差异的非对称量。
$\pi_{\theta_{old}} / \pi_{\theta}$	Old / New Policy	旧策略 / 新策略	前者采集当前 rollout, 后者是本轮正在优化的策略。
$J(\pi)$	Expected-return Objective	期望回报目标	策略 π 从初始状态出发的期望累计折扣回报。
$d_{\pi}(s)$	Discounted State-visitation Distribution	折扣状态访问分布	策略 π 访问各状态的折扣频率。
$A_{\pi}(s,a)$	Advantage Function	优势函数	动作 a 相对策略 π 在状态 s 下平均行为好多少。
$L_{\pi_{old}}(\pi)$	Surrogate Objective	代理目标	用旧策略状态分布近似评价候选新策略。
$\rho_t(\theta)$	Probability Ratio	概率比	新旧策略对已采样动作的概率或密度之比。
$D_{KL}(p q)$	KL Divergence	KL 散度	从分布 p 到分布 q 的相对差异。
δ_{KL}	KL Radius	KL 半径	TRPO 允许的平均 KL 上限。
g	Policy-objective Gradient	策略目标梯度	旧参数处代理目标的一阶梯度向量。
F	Fisher Information Matrix	Fisher 信息矩阵	KL 在旧参数附近的局部曲率矩阵。
$x=F^{-1}g$	Natural-gradient Direction	自然梯度方向	按策略分布曲率校正后的上升方向。
ϵ_{clip}	PPO Clipping Range	PPO 截断范围	PPO 允许 ratio 获得额外收益的区间半宽。
$\hat{A}_t$	Advantage Estimate	优势估计	由第 8 章的 GAE 等方法从 rollout 估计出的 Actor 权重。
B_{roll}	Rollout Batch	采样批次	由旧策略刚采集、本轮有限复用的 on-policy 数据。
$H(\pi)$	Policy Entropy	策略熵	衡量动作分布随机程度的量。

9.2 本章目标

学完本章，你应该能从头回答：

1. 为什么普通 Actor-Critic 不能无约束地在同一批 rollout 上更新很多次。
2. TRPO 的 surrogate objective 和 KL constraint 分别从哪里来。
3. TRPO 怎样把约束问题近似成“线性目标 + 二次约束”。
4. Natural Gradient、Fisher-vector Product、Conjugate Gradient 和 Line Search 各自做什么。
5. PPO clipping 为什么能近似限制更新，以及它没有严格保证什么。
6. 如何从一批 rollout 计算 GAE、ratio、Actor loss、Critic loss 和诊断指标。

9.3 本章主线

第 8 章已经得到 Actor-Critic：Critic 产生 advantage estimate，Actor 用它更新策略。但还留下一个优化问题：

rollout 是旧策略采集的。新策略更新越多，这批数据就越不能代表新策略真正会访问的状态和动作。

本章沿着下面的因果链展开：

Actor-Critic 可以更新 policy

- > 旧 rollout 只能局部评价新 policy
- > importance ratio 修正同一状态下的动作分布变化
- > 状态访问分布仍会随 policy 改变
- > TRPO 用 KL trust region 限制新旧策略距离
- > TRPO 的约束求解较复杂
- > PPO 用 clipped surrogate objective 近似保守更新

TRPO 不是 PPO 前面的历史背景。它给出了“为什么要限制策略更新”的完整数学和算法答案；理解 TRPO 后，PPO 的 ratio、clipping、multiple epochs 才能连成一条主线。

9.4 为什么旧策略数据只能局部评价新策略

9.4.1 策略一变，动作分布和状态分布都会变

旧策略 $\pi_{\theta_{old}}$ 产生一条轨迹：

$S_0 \rightarrow A_0 \rightarrow S_1 \rightarrow A_1 \rightarrow \dots$

更新策略后会同时发生两件事：

1. 在同一状态 s 下，新策略选择动作的概率改变。
2. 动作改变后，未来访问哪些状态的分布也改变。

第一种变化可以用 action probability ratio 修正；第二种变化涉及整条轨迹的反馈效应，单个时间步的 ratio 无法完全修正。

9.4.2 Performance Difference：新旧策略差多少

Performance Difference Lemma（性能差异引理）把两个策略的期望回报差写成：

$$J(\pi) - J(\pi_{old}) = \frac{1}{1-\gamma} \mathbb{E}_{s \sim d_{\pi}, a \sim \pi} [A_{\pi_{old}}(s, a)]$$

逐项解释：

- $J(\pi) - J(\pi_{old})$ ：候选新策略相对旧策略的真实改进。
- $A_{\pi_{old}}(s, a)$ ：用旧策略价值尺度评价动作 a 。
- $d_{\pi}(s)$ ：候选新策略真正会访问的状态分布。
- $1/(1-\gamma)$ ：把折扣状态访问频率还原为累计尺度。

这个式子的直觉是：如果新策略在自己会访问的状态上，经常选择旧策略认为 advantage 为正的的动作，新策略就会更好。

困难也恰好在这里：我们还没执行新策略，因此没有 d_{π} 下的数据。

9.4.3 Surrogate Objective: 先冻结旧状态分布

TRPO 用旧策略状态分布替换未知的新状态分布：

$$L_{\pi_{old}}(\pi) = J(\pi_{old}) + 1 / (1-\gamma) \mathbb{E}_{s \sim d_{\pi_{old}}, a \sim \pi} [A_{\pi_{old}}(s, a)]$$

这叫 Surrogate Objective (代理目标)：它在旧策略附近近似新策略回报，但不是对任意远的新策略都准确。

rollout 中动作也是按旧策略采样的，因此再用 Importance Sampling (重要性采样) 把 $a \sim \pi$ 改写成 $a \sim \pi_{old}$ ：

$$\rho_t(\theta) = \pi_{\theta}(A_t | S_t) / \{\pi_{\theta_{old}}(A_t | S_t)\}$$

$$L^{sur}(\theta) = \mathbb{E}_{B_{roll}} [\rho_t(\theta) \hat{A}_t]$$

其中 $\mathbb{E}_{B_{roll}}$ 表示对当前 rollout batch 中全部有效 transition 取样本平均。

ratio 的意义：

- $\rho_t = 1$ ：新策略对该动作的概率未变。
- $\rho_t > 1$ ：新策略更偏好该动作。
- $\rho_t < 1$ ：新策略降低该动作概率。

9.4.4 为什么有 Ratio 仍然不够

ratio 只在 rollout 已出现的状态 S_t 上修正动作概率。它没有完整回答：新策略变化后会不会进入旧 rollout 从未覆盖的状态。

如果新策略离旧策略很近，则两者状态访问分布也通常比较接近，代理目标更可信；如果一步更新很大， $d_{\pi_{\theta}}$ 与 $d_{\pi_{old}}$ 可能迅速分离，继续最大化旧数据上的代理目标不一定提升真实回报。

因此需要一个局部有效范围：

只在新旧策略分布足够接近的区域内，相信旧 rollout 给出的 surrogate improvement。

这个区域就是 trust region。

9.5 TRPO: 在 KL Trust Region 内优化代理目标

9.5.1 为什么用 KL 而不是参数欧氏距离

直接限制参数距离 $\|\theta - \theta_{old}\|$ 不可靠：

- 神经网络不同参数方向对输出分布的敏感度不同。
- 参数只变化很小，softmax 或 Gaussian 分布也可能变化很大。

- 参数变化较大，也可能因为冗余参数而几乎不改变策略。

TRPO 直接限制动作分布变化，使用平均 KL：

$$\overline{D_{KL}}(\theta_{old}, \theta) = \mathbb{E}_{S_t \sim d_{\pi_{old}}} [D_{KL}(\pi_{\theta_{old}}(\cdot | S_t) \parallel \pi_{\theta}(\cdot | S_t))]$$

对于离散动作：

$$D_{KL}(p \parallel q) = \sum_a p(a) \log p(a) / q(a)$$

KL 等于 0 表示两个分布相同；越大表示新策略偏离越明显。KL 不对称，因此必须写清方向。TRPO 的约束通常使用 old-to-new KL。

9.5.2 TRPO 的约束优化问题

TRPO 的核心形式为：

$$\begin{aligned} & \max_{\theta} \mathbb{E}_{B_{roll}} [\rho_t(\theta) \hat{A}_t] \\ & \text{subject to } \mathbb{E}_{B_{roll}} [D_{KL}(\pi_{\theta_{old}}(\cdot | S_t) \parallel \pi_{\theta}(\cdot | S_t))] \leq \delta_{KL} \end{aligned}$$

目标和约束职责不同：

部分	回答的问题
$\rho_t \hat{A}_t$	策略应该朝哪个方向改变，才能更偏好好动作、远离差动作？
平均 KL 约束	这一轮最多允许改变多少，旧 rollout 仍可被信任？

TRPO 并不是给普通 Policy Gradient 随便加一个 KL loss。它把策略更新明确写成 constrained optimization（约束优化）。

9.5.3 局部近似：线性目标加二次约束

直接对大型神经网络求解上述约束问题很难。TRPO 在旧参数 θ_{old} 附近做局部近似，记参数改变量：

$$\Delta\theta = \theta - \theta_{old}$$

代理目标做一阶展开：

$$L^{sur}(\theta) \approx L^{sur}(\theta_{old}) + g^T \Delta\theta$$

其中：

$$g = \nabla_{\theta} L^{sur}(\theta) |_{\theta=\theta_{old}}$$

平均 KL 在旧参数处的一阶导数为 0，因此保留二阶项：

$$\overline{D_{KL}} \approx 1 / 2 \Delta\theta^T F \Delta\theta$$

F 是 Fisher Information Matrix（Fisher 信息矩阵），在这里等价于平均 KL 对参数的局部二阶曲率。

于是原问题近似为：

$$\begin{aligned} \max_{\Delta\theta} & g^T \Delta\theta \\ \text{subject to} & 1 / 2 \Delta\theta^T F \Delta\theta \leq \delta_{KL} \end{aligned}$$

这一步是理解 TRPO 的核心：普通梯度用参数空间的欧氏长度衡量步子；TRPO 用策略分布的局部 KL 曲率衡量步子。

9.5.4 从约束问题得到 Natural-gradient Direction

为约束引入 Lagrange Multiplier (拉格朗日乘子) $\lambda_{TR} > 0$ ：

$$L(\Delta\theta, \lambda_{TR}) = g^T \Delta\theta - \lambda_{TR} (1 / 2 \Delta\theta^T F \Delta\theta - \delta_{KL})$$

对 $\Delta\theta$ 求导并令其为 0：

$$g - \lambda_{TR} F \Delta\theta = 0$$

因此方向满足：

$$\Delta\theta \propto F^{-1} g$$

定义：

$$x = F^{-1} g$$

x 就是 Natural-gradient Direction (自然梯度方向)。它会缩小那些“参数稍变就让策略分布剧烈变化”的方向，放大对策略分布较不敏感的方向。

再把步长缩放到 KL 边界：

$$\Delta\theta_{TRPO} = \sqrt{(2\delta_{KL} / \{x^T F x\})} x$$

代回可验证：

$$1 / 2 \Delta\theta_{TRPO}^T F \Delta\theta_{TRPO} = \delta_{KL}$$

9.5.5 为什么不能直接求 F^{-1}

如果策略网络有百万参数， F 是百万乘百万矩阵：

- 无法显式存储。
- 无法直接求逆。
- 即使近似构造，计算和数值误差也很大。

TRPO 不真正计算 F^{-1} ，而是求线性方程：

$$F x = g$$

Conjugate Gradient，简称 CG (共轭梯度法)，可以只通过反复计算 $F v$ 来近似求解 x ，不需要显式构造 F 。

9.5.6 Fisher-vector Product 怎么算

Fisher-vector Product, 简称 FVP (Fisher 矩阵向量积), 指给定任意向量 v , 计算:

$$Fv$$

因为 F 来自平均 KL 的二阶导数, 可以用自动微分计算:

```
mean_kl = mean(KL(old_policy || current_policy))
grad_kl = gradient(mean_kl, theta)
dot      = sum(grad_kl * v)
Fv       = gradient(dot, theta)
```

通常还会加入 damping (阻尼):

$$Fv \leftarrow Fv + c_{damp}v$$

$c_{damp} > 0$ 是 Damping Coefficient (阻尼系数), 用于改善数值条件, 避免曲率矩阵接近奇异时 CG 不稳定。

9.5.7 为什么有理论步长还要 Line Search

前面的步长来自两个局部近似:

1. surrogate objective 被线性化。
2. KL 被二次化。

神经网络不是严格线性/二次函数, 有限 batch 也有噪声。因此候选步 $\Delta\theta_{TRPO}$ 在真实计算中可能:

- 实际 KL 超过 δ_{KL} 。
- surrogate objective 没有改善。

Backtracking Line Search (回溯线搜索) 从完整候选步开始, 逐步缩小:

```
for fraction in [1, 0.5, 0.25, 0.125, ...]:
    theta_candidate = theta_old + fraction * full_step
    if mean_KL <= delta_KL and surrogate improves:
        accept candidate
        break
otherwise:
    keep theta_old
```

因此 TRPO 的完整稳定机制不是只有 KL 公式:

```
natural-gradient direction
+ KL-scaled candidate step
+ real surrogate/KL line-search validation
```

9.5.8 TRPO 一轮训练的完整流程

1. 用 `pi_old` 采集 on-policy rollout
2. 用 Critic 和 GAE 计算 advantage estimate
3. 固定 old log probability 和 old policy distribution
4. 计算 surrogate gradient g
5. 用 FVP + CG 近似求 $x = F^{-1}g$
6. 按 KL radius 缩放成 full step
7. 回溯线搜索：检查真实 surrogate 是否改善、真实 KL 是否合格
8. 接受候选参数，否则保留旧参数
9. 单独回归 Critic

Actor 的约束更新和 Critic 的 value regression 是两条优化线。TRPO 的 KL constraint 约束的是 Actor policy，不是 Critic value network。

9.5.9 TRPO 数值例子：二维参数怎样按曲率缩放

假设旧参数处：

$$g=(1,0.5)^T,$$

$$F=diag(2,1),$$

$$\delta_{KL}=0.01$$

第一步，求自然梯度方向：

$$x=F^{-1}g=(0.5,0.5)^T$$

第一个参数方向的曲率是 2，比第二个方向更敏感，因此同样梯度下被缩小得更多。

第二步，计算：

$$x^T F x = 0.75$$

第三步，缩放系数：

$$c_{step} = \sqrt{(2 \times 0.01 / 0.75)} \approx 0.1633$$

最终候选步：

$$\Delta\theta = c_{step} x \approx (0.0816, 0.0816)^T$$

局部预测 KL 为：

$$1 / 2 \Delta\theta^T F \Delta\theta \approx 0.01$$

实际训练还必须用新参数重新计算真实 KL 和 surrogate，再由 line search 决定是否接受。

9.6 PPO：把 Trust-region 思想改成一阶可优化目标

9.6.1 为什么还需要 PPO

TRPO 的逻辑完整，但工程实现成本高：

- 需要二阶自动微分得到 FVP。

- 需要 CG 迭代求自然梯度方向。
- 需要 line search 多次前向评估。
- 与普通 minibatch optimizer 的接口不完全一致。

PPO 希望保留“不要离旧策略太远”的思想，同时只使用普通一阶梯度下降。

9.6.2 PPO Clipped Objective

先定义未截断代理项：

$$u_t(\theta) = \rho_t(\theta) \hat{A}_t$$

再把 ratio 限制到：

$$\tilde{\rho}_t(\theta) = \text{clip}(\rho_t(\theta), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}})$$

PPO 要最大化：

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_{B_{\text{roll}}} [\min(\rho_t \hat{A}_t, \tilde{\rho}_t \hat{A}_t)]$$

自动微分框架通常最小化 loss，因此实现写成：

$$L_{\text{policy}} = -L^{\text{CLIP}}$$

9.6.3 min 为什么同时处理正负 Advantage

不要只背“ratio 截断到 $[0.8, 1.2]$ ”。PPO 的行为取决于 advantage 的符号：

Advantage	策略想做什么	何时进入平台区
$\hat{A}_t > 0$	提高已采样好动作的概率	$\rho_t > 1 + \epsilon_{\text{clip}}$
$\hat{A}_t < 0$	降低已采样差动作的概率	$\rho_t < 1 - \epsilon_{\text{clip}}$

当 $\hat{A}_t > 0$ ，ratio 越大， $\rho_t \hat{A}_t$ 越大；超过上界后 clipped 项不再增加，**min** 选择平台值。

当 $\hat{A}_t < 0$ ，ratio 越小， $\rho_t \hat{A}_t$ 反而越大；低于下界后 clipped 项更负，**min** 选择这个保守值，继续降低概率不再改善目标。

PPO clipping 是**单边保守**的：

- 好动作若概率错误地下降，仍有梯度把它拉回去。
- 差动作若概率错误地上升，仍有梯度把它压回去。
- 只有沿正确方向走得太远，额外收益才被截住。

9.6.4 PPO Clip 的不是实际 Policy

PPO 没有把网络输出或 ratio 强行投影进区间。它只截断 surrogate objective 中继续朝有利方向移动的收益。

多个样本共享网络参数，因此一个 minibatch 更新后：

- 某些样本 ratio 可能大于 $1 + \epsilon_{\text{clip}}$ 。
- 某些样本 ratio 可能小于 $1 - \epsilon_{\text{clip}}$ 。
- 平均 KL 仍可能超过期望范围。

所以 PPO 不是严格 trust region。工程上通常同时监控 Approximate KL（近似 KL）和 Clip Fraction（截断比例）。

9.6.5 PPO 为什么能复用同一批 Rollout 几个 Epoch

刚采样完成时：

$$\theta = \theta_{old}, \rho_t = 1$$

随着对同一批数据做 minibatch 更新， ρ_t 逐渐偏离 1。clipping 会让已经沿正确方向变化过大的样本停止提供额外收益，从而降低同一批旧数据被过度利用的风险。

但复用是有限的：

- epoch 太多，新策略和 rollout policy 仍会分离。
- clipping 不能修复旧 rollout 缺失的新状态。
- 下一轮仍要用更新后的策略重新采样。

因此 PPO 仍是 on-policy，而不是因为有 ratio 就变成普通 off-policy replay 算法。

9.6.6 Actor、Critic 与 Entropy 的完整 Loss

Policy loss：

$$L_{policy} = -\mathbb{E} [\min(\rho_t \hat{A}_t, \tilde{\rho}_t \hat{A}_t)]$$

Value loss：

$$L_{value} = \mathbb{E} [(V_w(S_t) - \hat{V}_t^{target})^2]$$

Entropy bonus 对应的最小化项：

$$L_{entropy} = -\beta_H \mathbb{E} [H(\pi_\theta(\cdot | S_t))]$$

总 loss 常写成：

$$L_{total} = L_{policy} + c_V L_{value} + L_{entropy}$$

其中 c_V 是 Value-loss Coefficient (价值损失系数)， β_H 是 Entropy Coefficient (熵系数)。

若 Actor 和 Critic 共享 backbone，总 loss 会共同更新共享参数；若两个网络分离，则分别反向传播更清晰。

9.6.7 常见稳定化与诊断指标

项目	含义	异常时说明什么
Advantage Normalization	batch 内对 $\hat{A}$ 减均值、除标准差	不做时 reward 尺度会直接影响 Actor 梯度尺度。
Gradient Clipping	限制整体梯度范数	极端 batch 不至于产生巨大参数步。
Value Clipping	限制新旧 value prediction 变化	Critic 一次更新过猛时提供保护。
Approximate KL	估计新旧策略平均差异	过大表示 clipping 没能充分限制更新。
Clip Fraction	有多少样本进入 clipping 平台	长期过高可能表示学习率或 epoch 数过大。
Explained Variance	Critic 对 return 波动的解释程度	很低或为负说明 Critic 信号不可靠。
Entropy	当前策略随机程度	过快降到很低可能导致探索过早消失。

target KL early stopping 的做法是：若某个 epoch 后实测 approximate KL 超过阈值，提前停止继续复用这批 rollout。

9.7 两个完整数值例子

9.7.1 例子一：正 Advantage 被上界截断

已知：

```
old probability = 0.40
new probability = 0.52
advantage      = +2
epsilon_clip   = 0.20
```

ratio：

$$\rho = 0.52 / 0.40 = 1.3$$

允许区间是 $[0.8, 1.2]$ 。未截断项：

$$\rho \hat{A} = 1.3 \times 2 = 2.6$$

截断项：

$$\text{clip}(1.3, 0.8, 1.2) \times 2 = 1.2 \times 2 = 2.4$$

PPO 最大化两者较小值：

$$\min(2.6, 2.4) = 2.4$$

这个好动作的概率已经提升过多，继续从 0.52 往上增加不会再提高本样本的 clipped objective。

9.7.2 例子二：负 Advantage 被下界截断

已知：

```
old probability = 0.30
new probability = 0.21
advantage      = -3
epsilon_clip   = 0.20
```

ratio：

$$\rho = 0.21 / 0.30 = 0.7$$

未截断项：

$$\rho \hat{A} = 0.7 \times (-3) = -2.1$$

截断项：

$$0.8 \times (-3) = -2.4$$

PPO 取：

$$\min(-2.1, -2.4) = -2.4$$

虽然 ratio 继续减小会让未截断项从负数变得更大，但 `min` 固定选择 -2.4，所以过度压低差动作概率不会获得额外收益。

9.7.3 例子三：Categorical Policy 的 KL

旧策略和新策略对三个动作的分布分别为：

$$p=[0.5, 0.3, 0.2], q=[0.6, 0.25, 0.15]$$

old-to-new KL 为：

$$\begin{aligned} D_{KL}(p||q) &= 0.5\log(0.5/0.6) \\ &+ 0.3\log(0.3/0.25) \\ &+ 0.2\log(0.2/0.15) \\ &\approx 0.021 \end{aligned}$$

TRPO 会把多个 rollout 状态上的 KL 取平均，再与 δ_{KL} 比较。单个样本 action ratio 合理，不代表整个动作分布的 KL 一定小。

9.8 PPO 一轮训练的完整闭环

```
initialize actor theta and critic w

repeat:
  1. freeze theta_old <- theta
  2. use pi_theta_old to collect rollout batch
  3. save state, action, reward, done, old_log_prob, old_value
  4. bootstrap at non-terminal rollout boundaries
  5. compute TD residuals backward
  6. compute GAE advantages and value targets
  7. optionally normalize advantages

  for several epochs:
    shuffle rollout batch into minibatches
    new_log_prob = log pi_theta(action | state)
    ratio = exp(new_log_prob - old_log_prob)
    policy_loss = negative clipped objective
    value_loss = regression to fixed value target
    entropy_loss = negative entropy bonus
    update parameters with minibatch gradient descent

    measure approximate KL and clip fraction
    stop current epochs early if KL is too large

  discard rollout batch
```

关键梯度边界：

- `old_log_prob` 是采样时保存的常量，不反向传播。
- GAE advantage 和 value target 在本轮优化中固定。
- Actor loss 不沿 advantage 回到 Critic。
- 下一轮必须重新采样，不能把旧 rollout 长期留在 replay buffer。

9.9 TRPO 与 PPO 的统一比较

维度	TRPO	PPO-Clip
基本目标	最大化 surrogate	最大化 clipped surrogate
更新限制	显式平均 KL constraint	通过 ratio 平台间接限制
优化方法	FVP + CG + line search	普通 minibatch SGD/Adam
是否严格满足 trust region	line search 后更接近显式约束	不保证
数据使用	on-policy rollout	on-policy rollout, 有限多 epoch
实现复杂度	高	低
常见诊断	KL、surrogate improvement、CG residual	KL、clip fraction、entropy、value quality

一句话总结：

TRPO 明确求解“最大化局部代理目标，同时限制策略 KL”；PPO 不再求自然梯度和线搜索，而是让超出安全范围的有利 ratio 变化不再获得额外收益，从而近似保守更新。

9.10 场景例子

9.10.1 机器人连续控制

策略输出关节力矩 Gaussian distribution。一次过大的策略更新可能让均值和标准差同时剧烈变化，旧 rollout 中安全的动作区域不再代表新策略。

TRPO 会限制整组 Gaussian policy 的平均 KL；PPO 则保存 rollout 时动作的 old log probability，通过 ratio clipping 做多轮 minibatch 更新。两者都比完全无约束地重复优化旧 rollout 更稳定。

9.10.2 RLHF 中的 Token Policy

语言模型每一步输出 token categorical distribution。PPO rollout 包含 prompt、生成 token、old log probability、reward、value 和 advantage。

需要区分两个限制：

- PPO ratio clipping：限制当前训练策略相对 rollout policy 的单轮更新。
- 对 reference model 的 KL penalty：限制对齐后的策略长期偏离 SFT/reference model。

它们不是同一个 KL，也不承担同一职责。

9.11 面试高频问题

9.11.1 为什么普通 Policy Gradient 更新过大可能崩溃？

因为 rollout 来自旧策略。新策略变化后，动作分布和将来状态访问分布都会变化；若离旧策略太远，旧数据上的 surrogate improvement 不再可靠。

9.11.2 TRPO 的核心优化问题是什么？

最大化旧 rollout 上的重要性采样 surrogate objective，同时约束新旧策略的平均 KL 不超过给定半径。

9.11.3 Natural Gradient 为什么是 $F^{-1}g$ ？

普通梯度只考虑目标的一阶增益；Fisher 矩阵描述参数变化对策略分布 KL 的局部曲率。 $F^{-1}g$ 会按分布敏感度校正梯度方向，在相同局部 KL 成本下争取更大的代理目标增益。

9.11.4 CG、FVP 和 Line Search 分别做什么？

- FVP：不显式构造 Fisher 矩阵，计算 Fv 。
- CG：仅依赖 FVP，近似求解 $Fx=g$ 。
- Line Search：验证局部近似得到的候选步在真实网络上确实改善 surrogate 且满足 KL。

9.11.5 PPO 为什么仍是 On-policy？

它只有限复用刚由旧策略采集的 rollout。clipping 让少量 epoch 更安全，但不能修复长期历史数据和当前策略之间的状态分布偏移；每轮更新后仍要重新采样。

9.11.6 PPO Clip 是否保证 Ratio 一定在区间内？

不保证。它 clip 的是代理目标中的收益，不是把策略投影到区间。共享参数更新后仍可能有 ratio 越界，因此实践中还要监控 KL 和 clip fraction。

9.12 原始论文与参考资料

1. [Trust Region Policy Optimization, Schulman et al., 2015](#)
2. [Proximal Policy Optimization Algorithms, Schulman et al., 2017](#)

阅读建议：先用本章理解 surrogate、KL constraint 和局部求解，再看 TRPO 论文中的 monotonic-improvement bound；PPO 论文重点看 clipped surrogate、adaptive KL penalty 和实验设计。

9.13 本章练习

1. 从 Performance Difference 解释为什么 surrogate 要冻结旧状态分布。
2. 写出 TRPO 的目标和 KL constraint，并说明两部分职责。
3. 说明为什么 KL 在旧参数附近使用二阶近似。
4. 已知 g 、对角 Fisher 矩阵和 KL 半径，计算 TRPO 候选步。
5. 分别计算正 advantage 和负 advantage 下的 PPO clipped objective。
6. 解释 PPO 为什么能复用一批 rollout 几个 epoch，却仍不是 off-policy replay。
7. 解释 PPO ratio clipping 与 RLHF reference KL penalty 的区别。

▼ 参考答案（点击展开）

1. 冻结旧状态分布：真实性能差使用新策略状态分布 d_π ，但新策略尚未执行，无法直接采样。用旧策略状态分布构造 surrogate 可以从现有 rollout 估计，但只在新旧策略足够接近时可靠，因此还需要 trust region。
2. TRPO 约束问题：目标是最大化 $E[\rho_r \hat{A}_t]$ ，决定更新方向；约束是平均 $D_{KL}(\pi_{old} \parallel \pi_\theta) \leq \delta_{KL}$ ，限制更新幅度。
3. KL 二阶近似：在 $\theta = \theta_{old}$ 时新旧分布相同，KL 为 0 且一阶导数为 0，所以局部主要变化来自二阶曲率，写成 $1/2A\theta^T F A\theta$ 。
4. 候选步：先解 $x = F^{-1}g$ ，再计算 $\Delta\theta = \sqrt{(2\delta_{KL}/(x^T F x))}x$ 。实际还要 line search 验证真实 KL 和 surrogate。
5. PPO 两种符号：正 advantage 时，ratio 超过上界后目标固定在 $(1+\epsilon)\hat{A}$ ；负 advantage 时，ratio 低于下界后目标固定在 $(1-\epsilon)\hat{A}$ 。 $\min$ 与 advantage 符号共同产生单边平台。

6. 有限复用但仍 **on-policy**：数据来自当前旧策略，ratio/clipping 只支持在其附近做有限更新。随着 epoch 增多，状态分布偏移仍会增大，因此 batch 用完即丢弃，下一轮重新采样。
7. 两个限制：PPO ratio 比较当前参数和本轮 rollout policy，限制一次优化迭代；reference KL 比较训练策略和固定 SFT/reference model，限制长期对齐漂移。